INVERTIS UNIVERSITY, BAREILLY

B.Tech CSE – Third Year

APPLIED DATA STRUCTURES & ALGORITHMS LAB (BCS553)

LAB 14

Assignment 14: Sorted Check, Search and Fibonacci

Assignment aim: Use recursion for sorted checks, searching and Fibonacci values.

Experiment No. 14.1 — Recursively check sorted array

1. Experiment Title: Recursively check sorted array

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Recursive checks and searches advance through an array; Fibonacci is defined using two preceding values.

5. Algorithm / Procedure:
If at the last element, return true; if a pair is out of order, return false; otherwise check the next pair recursively.
Then compile and execute the Java program.

6. Program:

```
public class Task_01_Recursively_check_sorted_array{
    static boolean isSorted(int[] a, int i) {
        if (i >= a.length - 1) return true;
        if (a[i] > a[i + 1]) return false;
        return isSorted(a, i + 1);
    }
    public static void main(String[] args) {
        int[] a = {2, 4, 6, 8, 10};
        System.out.println(isSorted(a, 0));
    }
}
```

7. Input: Fixed input values are declared in the program: the array/key/n value specified in the task.

8. Output:
true

Time / Space Complexity: O(n) time; O(n) call-stack space.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- What does ascending order mean?
- Why compare adjacent elements?
- When does the method return false?

Experiment No. 14.2 — Find index recursively

1. Experiment Title: Find index recursively

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Recursive checks and searches advance through an array; Fibonacci is defined using two preceding values.

5. Algorithm / Procedure:
Stop with -1 when the index reaches the array length; return i when the key matches; otherwise search from i + 1.
Then compile and execute the Java program.

6. Program:

```
public class Task_02_Find_index_recursively{
    static int findIndex(int[] a, int key, int i) {
        if (i == a.length) return -1;
        if (a[i] == key) return i;
        return findIndex(a, key, i + 1);
    }
    public static void main(String[] args) {
        int[] a = {45, 67, 80, 52, 90};
        System.out.println(findIndex(a, 80, 0));
    }
}
```

7. Input: Fixed input values are declared in the program: the array/key/n value specified in the task.

8. Output:
2

Time / Space Complexity: O(n) time; O(n) call-stack space.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- What does -1 mean?
- Does this recursive search require sorted input?
- What is returned when the key is found?

Experiment No. 14.3 — Calculate Fibonacci recursively

1. Experiment Title: Calculate Fibonacci recursively

2. Aim: Implement the task specified in the BCS553 lab assignment.

3. Software / Tools Required: Java JDK, a text editor or IDE (such as IntelliJ IDEA, Eclipse, or VS Code), and a terminal/command prompt.

4. Theory: Recursive checks and searches advance through an array; Fibonacci is defined using two preceding values.

5. Algorithm / Procedure:
Return 0 for fib(0) and 1 for fib(1); otherwise return fib(n - 1) + fib(n - 2).
Then compile and execute the Java program.

6. Program:

```
public class Task_03_Calculate_Fibonacci_recursively{
    static int fib(int n) {
        if (n == 0) return 0;
        if (n == 1) return 1;
        return fib(n - 1) + fib(n - 2);
    }
    public static void main(String[] args) {
        System.out.println(fib(7));
    }
}
```

7. Input: Fixed input values are declared in the program: the array/key/n value specified in the task.

8. Output:
13

Time / Space Complexity: O(2^n) time for this simple recursive version; O(n) maximum call-stack depth.

9. Result: The program implements the specified task and produces the expected output for the given test data.

10. Viva-Voce Questions:

- What are the first two Fibonacci values?
- What is the recurrence relation?
- Why is this version inefficient for large n?